22. Ejercicio de Mermaid

Jonathan Ricardo Serna Galina
20074902

Programación Asistida

1. Descripción

En este ejercicio modelé un sistema para reservar equipos de laboratorio por una hora, con aprobación de un administrador. Comparé una propuesta inicial con una versión corregida mediante diagramas de flujo, casos de uso y clases.

Organicé la actividad en una simulación de cuatro turnos con la IA: prompt inicial, primera propuesta, solicitud de cinco cambios y respuesta corregida. Conservé los códigos y las capturas de ambas versiones.

Modalidad: simulación directa realizada con ChatGPT/Codex. No ejecuté OpenCode. La revisión de fallos también fue asistida; debo revisarla personalmente para completar el paso que la consigna solicita sin IA.

Usé la portada de los PDFs de ejercicios como referencia de formato. Las capturas proceden de un visor local con Mermaid 12.2.0; no acreditan validación en mermaid.live.

Contenido

1. Descripción de la actividad.
2. Fallos, cinco cambios y criterios de revisión.
3. Conversación completa de la simulación, con código.
4. Seis capturas: antes y después de cada diagrama.
5. Validación y fuentes.

2. Fallos y cambios solicitados

Alumno: Jonathan Ricardo Serna Galina
Matrícula: 20074902
Materia: Programación Asistida
Ejercicio: 22. Mermaid - Reservas de laboratorio

Este análisis forma parte de una simulación asistida. La consigna pide una revisión «sin IA»; esta sección no acredita ese paso y debo revisarla personalmente antes de entregar.

Tres fallos de la primera versión

F1. El flujo permite aprobar dos reservas para el mismo equipo y horario

El flujo inicial pasa de solicitar a aprobar sin comprobar cruces. Aunque dice «una hora», tampoco deriva la hora de fin de la de inicio. Dos solicitudes de 10:00 a 11:00 podrían terminar aprobadas. El atributo `Equipment.available` tampoco representa disponibilidad por fecha: un equipo puede estar ocupado a las 10:00 y libre a las 11:00.

Esto falla porque el equipo no puede asignarse a dos personas al mismo tiempo. Necesito fijar `end = start + 1 hora`, exigir un inicio futuro y comparar intervalos del mismo equipo. Al aprobar debo repetir la comprobación y guardar de forma atómica para evitar aprobaciones simultáneas incompatibles.

F2. Los casos de uso permiten al usuario aprobar y no muestran el rechazo

En `antes/02_casos_uso.mmd`, la relación `User -- Approve` conecta al usuario común con una decisión administrativa. Además, falta «Rechazar reserva», aunque el flujo inicial sí contempla ese resultado.

Esto falla porque confunde los permisos y deja los diagramas en desacuerdo. Si alguien programa siguiendo el diagrama, podría permitir que una persona se apruebe su propia solicitud. Solo el administrador debe aprobar, rechazar y registrar asistencia; el usuario consulta, solicita y cancela sus propias reservas.

F3. Las clases dejan ambiguas las relaciones y los estados

Las asociaciones iniciales no muestran multiplicidades y `status` acepta cualquier texto. No queda claro si una reserva corresponde a uno o varios usuarios o equipos, ni cuántas reservas puede revisar un administrador. Tampoco se distinguen estados válidos de errores como escribir «aproavada».

Esto falla porque el modelo no define con precisión qué datos pueden existir. Necesito exactamente un usuario y un equipo por reserva, muchas reservas históricas por cada uno, un administrador revisor opcional mientras esté pendiente y un conjunto cerrado de estados.

Exactamente cinco cambios pedidos

1. **C1, corrige F1:** fijar la duración en una hora, exigir inicio futuro y equipo habilitado, impedir cruces con reservas aprobadas del mismo equipo y revalidar todo al aprobar mediante una operación atómica. Justificación: evitar reservas dobles, incluso con decisiones simultáneas.
2. **C2, corrige F2:** quitar la aprobación al usuario común y agregar el rechazo exclusivo del administrador. Mantener autenticación y permisos coherentes en los tres diagramas. Justificación: impedir la autoaprobación y representar ambas decisiones.
3. **C3, corrige F3:** agregar multiplicidades, un revisor opcional y estados enumerados. Justificación: definir sin ambigüedad la propiedad, el equipo asignado y el ciclo de vida.
4. **C4, regla nueva:** permitir que el propietario cancele una reserva pendiente o aprobada hasta 24 horas antes. Aceptar exactamente 24 horas; con menos tiempo se conserva el estado. Liberar el intervalo si estaba aprobado. Justificación: permitir la cancelación sin cambios de último momento.
5. **C5, regla nueva:** al registrar la tercera inasistencia, bloquear al usuario durante 30 días e impedir nuevas solicitudes y aprobaciones mientras el bloqueo esté vigente. Justificación: aplicar la consecuencia solicitada y evitar que el usuario la eluda con solicitudes pendientes.

Criterios que fijé para eliminar ambigüedades

- Los intervalos son `[inicio, fin)`: 10:00-11:00 y 11:00-12:00 no se cruzan. Hay cruce si `nuevoInicio < finExistente` y `nuevoFin > inicioExistente`.
- Solo las aprobadas ocupan el equipo. Puede haber solicitudes pendientes para el mismo horario; al aprobar una, la otra ya no puede aprobarse.
- Una solicitud pendiente puede pasar a aprobada, rechazada o cancelada. Una aprobada puede pasar a cancelada, completada o inasistencia. Estos últimos estados son finales.
- Una reserva tiene cero revisores antes de decidirse y uno después de una aprobación o un rechazo. El revisor se conserva si luego se cancela o se cierra. Si se cancela mientras está pendiente, puede conservar cero.
- Cada operación exige sesión y rol autorizado. La cancelación también verifica la propiedad; un usuario bloqueado sí puede consultar y cancelar reservas propias si cumple el plazo.
- El administrador registra la asistencia al terminar la hora. Solo una reserva aprobada puede cerrarse y solo una vez. Una cancelada o rechazada nunca genera una falta.
- Las tres inasistencias son acumuladas desde el último bloqueo, no necesariamente consecutivas. Al aplicar el bloqueo, el contador vuelve a cero y el historial permanece en las reservas.
- Los 30 días empiezan cuando el administrador registra la tercera falta. El bloqueo termina cuando `ahora >= blockedUntil`. Todas las fechas se comparan con la misma zona horaria del laboratorio.

- El bloqueo impide solicitar y aprobar; no cancela automáticamente reservas ya aprobadas. La consigna no define esa consecuencia adicional.
- Estos criterios completan detalles que la consigna no especifica; son decisiones de la simulación, no requisitos textuales del profesor.

Comprobaciones de escritorio

Caso	**Resultado esperado del modelo**
Elegir inicio 10:00	El fin se calcula como 11:00, sin duración libre.
Solicitar con inicio pasado o equipo deshabilitado	Denegar.
Equipo aprobado de 10:00 a 11:00; nueva de 10:30 a 11:30	Denegar por cruce.
Equipo aprobado de 10:00 a 11:00; nueva de 11:00 a 12:00	Permitir la solicitud.
Dos pendientes para un mismo equipo y hora	Solo una puede aprobarse; revalidar al guardar.
Usuario común intenta aprobar	Denegar por rol.
Propietario cancela exactamente 24 horas antes	Cancelar pendiente o aprobada.
Propietario cancela 23 h 59 min antes	Denegar y conservar estado.
Otra persona intenta cancelar	Denegar aunque falten más de 24 horas.
Intentar cerrar una aprobada antes del fin	Denegar.
Registrar tercera falta al terminar	Bloquear 30 días y reiniciar el contador.
Repetir el registro de esa misma falta	Denegar; no aumentar el contador.
Registrar asistencia a una cancelada	Denegar; no generar una falta.
Usuario con bloqueo vigente solicita o espera aprobación	Denegar solicitud o aprobación.
Se alcanza exactamente la fecha de desbloqueo	Permitir solicitar, sujeto a las demás validaciones.

Estas comprobaciones son recorridos del diseño, no pruebas de un sistema implementado. La validación de sintaxis y las capturas se documentan por separado.

3. Conversación con la IA

Ejercicio 22. Mermaid - Reservas de laboratorio
Alumno: Jonathan Ricardo Serna Galina, 20074902
Modalidad: simulación directa en ChatGPT/Codex; no se ejecutó OpenCode.

Este registro conserva completos los cuatro turnos de la simulación académica, incluidos los seis bloques Mermaid. Los mensajes del estudiante son un guion elaborado con asistencia a petición del usuario; no son mensajes humanos adicionales ni una exportación del historial completo de la aplicación. La primera respuesta es una propuesta inicial con limitaciones para comparar el antes y el después.

Turno 1. Estudiante - Prompt inicial de la consigna

Hazme los diagramas de flujo, casos de uso y clases en mermaid de un sistema de reservas de laboratorio. Los usuarios reservan equipos por una hora y el admin aprueba las reservas. Los diagramas deben estar completos y seguir buenas prácticas. Entrégamelos en código para que pueda pegarlos en un visor.

Turno 2. IA - Primera versión

Propongo tres diagramas: un flujo desde la solicitud hasta la decisión del administrador, los casos de uso de los actores y las clases principales. Cada bloque se pega por separado en un visor. Para el diagrama nativo de casos de uso se necesita Mermaid 12 o posterior.

Diagrama de flujo - Antes

```
flowchart TD
    Start([Inicio]) --> Login[Iniciar sesión]
    Login --> Select[Elegir equipo y horario]
    Select --> Request[Solicitar reserva de una hora]
    Request --> Pending[Guardar como pendiente]
    Pending --> Review{¿El administrador aprueba?}
    Review -->|Sí| Approved[Marcar como aprobada]
    Review -->|No| Rejected[Marcar como rechazada]
    Approved --> Use[Usar el equipo]
    Use --> Finish([Fin])
    Rejected --> Finish
```

Casos de uso - Antes

```
usecase-beta
direction LR
```

```
actor User("Usuario")
actor Admin("Administrador")
systemBoundary Lab["Sistema de reservas de laboratorio"]
    Browse("Consultar equipos")
    Request("Solicitar reserva de una hora")
    View("Consultar mis reservas")
    Approve("Aprobar reserva")
end
User -- Browse
User -- Request
User -- View
User -- Approve
Admin -- Approve
```

Diagrama de clases - Antes

```
classDiagram
    class User {
        +int id
        +string name
        +requestReservation()
    }
    class Administrator {
        +int id
        +string name
        +approveReservation()
    }
    class Equipment {
        +int id
        +string name
        +bool available
    }
    class Reservation {
        +int id
        +datetime start
        +datetime end
        +string status
    }
    User --> Reservation : solicita
    Administrator --> Reservation : aprueba
    Reservation --> Equipment : utiliza
```

Turno 3. Estudiante - Revisión y cinco cambios concretos

Al revisar la propuesta encontré tres problemas: el flujo no revisa choques de horario, el usuario aparece con permiso para aprobar y el diagrama de clases no indica multiplicidades ni estados definidos. Los justifico por separado en `fallos_y_cambios.md`. Te pido exactamente estos cinco cambios:

1. Fija cada reserva en una hora, con inicio futuro y equipo habilitado. Evita cruces con aprobadas del mismo equipo y revalida al aprobar en una operación atómica.
2. Quita al usuario común el permiso de aprobar y agrega el rechazo exclusivo del administrador. Mantén los permisos coherentes en los tres diagramas.

3. Completa las clases con multiplicidades, un administrador revisor opcional y estados enumerados para las reservas.
4. Agrega la cancelación de una reserva propia pendiente o aprobada hasta 24 horas antes. Acepta exactamente 24 horas y libera el horario si estaba aprobado.
5. Agrega que tres inasistencias bloqueen al usuario durante 30 días. Registra cada falta una sola vez e impide solicitar y aprobar durante el bloqueo.

Los primeros tres corrigen los fallos que señalé. Los últimos dos agregan las reglas que pide la actividad. Actualiza los tres diagramas y conserva los originales.

Turno 4. IA - Versión corregida

Apliqué los cinco cambios. La solicitud queda pendiente hasta una decisión autorizada. La disponibilidad se calcula por equipo y horario, y se vuelve a comprobar al aprobar. Los casos de uso separan los permisos de los actores; las clases indican relaciones, estados y las operaciones que aplican las reglas.

En los casos de uso, `include` indica una validación obligatoria y `extend` una conducta condicional: registrar una falta solo cuando el usuario no asistió y bloquear solo al acumular la tercera. La autenticación es una precondición de todas las operaciones.

Diagrama de flujo - Después

```
flowchart LR
    subgraph Request["1. Solicitud del usuario"]
        direction TB
        R0([Solicitar reserva]) --> R1{"¿Sesión válida y<br/>sin bloqueo vigente?"}
        R1 -->|No| R2[Denegar solicitud]
        R1 -->|Sí| R3[Elegir equipo e inicio futuro<br/>Fin = inicio + 1 hora]
        R3 --> R4{"¿Equipo habilitado y<br/>sin cruce con aprobadas?"}
        R4 -->|No| R5[Informar indisponibilidad]
        R4 -->|Sí| R6[Guardar PENDIENTE<br/>Avisar al administrador]
    end
    subgraph Review["2. Decisión del administrador"]
        direction TB
        A0([Admin autenticado:<br/>seleccionar pendiente]) --> A1{¿Aprobar?}
        A1 -->|No| A2[RECHAZADA y notificar]
        A1 -->|Sí| A3{"Revalidar al guardar:<br/>sigue pendiente, inicio futuro,<br/>usuario sin bloqueo,
<br/>equipo habilitado y sin cruce"}
        A3 -->|Sí| A4[APROBADA y notificar<br/>Ocupar intervalo de forma atómica]
        A3 -->|No| A5[No aprobar y notificar<br/>Si sigue pendiente, rechazar]
    end
    subgraph Cancel["3. Cancelación por el propietario"]
        direction TB
        C0([Usuario autenticado:<br/>cancelar reserva propia]) --> C1{"¿PENDIENTE o APROBADA<br/>y faltan al
menos 24 horas?"}
        C1 -->|Sí| C2[CANCELADA y notificar<br/>Liberar intervalo si estaba aprobado]
        C1 -->|No| C3[Denegar cancelación<br/>Conservar estado]
    end
    subgraph Attendance["4. Cierre y bloqueo"]
```

```
        direction TB
        T0([Admin autenticado:<br/>al terminar la hora]) --> T1{"¿APROBADA y<br/>aún sin cierre?"}
        T1 -->|No| T2[No registrar asistencia<br/>ni duplicar inasistencias]
        T1 -->|Sí| T3{¿El usuario asistió?}
        T3 -->|Sí| T4[COMPLETADA]
        T3 -->|No| T5[INASISTENCIA<br/>Sumar una al contador]
        T5 --> T6{¿Contador = 3?}
        T6 -->|No| T7[Conservar contador]
        T6 -->|Sí| T8[Bloquear por 30 días desde el registro<br/>Reiniciar contador a 0 y notificar]
    end
    %% Invisible links only arrange the four independent entry points.
    Request ~~~ Review
    Review ~~~ Cancel
    Cancel ~~~ Attendance
```

Casos de uso - Después

```
---
config:
  layout: elk
---
usecase-beta
direction LR
actor User("Usuario")
actor Admin("Administrador")
systemBoundary Lab["Sistema de reservas de laboratorio"]
    Browse("Consultar disponibilidad")
    Request("Solicitar reserva de una hora")
    ValidateRequest("Validar usuario habilitado, inicio futuro y equipo libre")
    View("Consultar mis reservas")
    Cancel("Cancelar reserva propia")
    ValidateCancel("Validar pendiente o aprobada y plazo mínimo de 24 h")
    Approve("Aprobar reserva pendiente")
    ValidateApproval("Revalidar condiciones y guardar atómicamente")
    Reject("Rechazar reserva pendiente")
    Attendance("Registrar asistencia al terminar")
    Absence("Registrar falta de aprobada una sola vez")
    Block("Bloquear 30 días al acumular 3 faltas y reiniciar contador")
end
User -- Browse
User -- Request
User -- View
User -- Cancel
Admin -- Approve
Admin -- Reject
Admin -- Attendance
Request ..> : include ValidateRequest
Cancel ..> : include ValidateCancel
Approve ..> : include ValidateApproval
Absence ..> : extend Attendance
Block ..> : extend Absence
%% ValidateRequest also checks equipment enabled and conflicts with approved slots.
%% ValidateApproval repeats all request guards while the reservation is pending.
%% Absence extends Attendance only if the user did not attend.
```

Diagrama de clases - Después

```
classDiagram
    direction TB
    class User {
        +int id
        +string name
        +int absenceCount
        +datetime blockedUntil
        +isBlocked(now) bool
    }
    class Administrator {
        +int id
        +string name
    }
    class Equipment {
        +int id
        +string name
        +bool enabled
    }
    class Reservation {
        +int id
        +datetime start
        +datetime end
        +ReservationStatus status
    }
    class ReservationStatus {
        <<enumeration>>
        PENDING
        APPROVED
        REJECTED
        CANCELLED
        COMPLETED
        NO_SHOW
    }
    class ReservationService {
        +request(user, equipment, start) Reservation
        +approve(admin, reservation, now)
        +reject(admin, reservation)
        +cancel(owner, reservation, now)
        +close(admin, reservation, attended, now)
    }
    User "1" --> "0..*" Reservation : solicita
    Equipment "1" --> "0..*" Reservation : recibe en distintos horarios
    Administrator "0..1" --> "0..*" Reservation : decide
    Reservation --> ReservationStatus : estado
    ReservationService ..> Reservation : valida y cambia estado
    note for Reservation "end = start + 1 h; inicio futuro.<br/>Intervalo [start, end); sin cruce de
aprobadas por equipo.<br/>Cancelar propia PENDING o APPROVED<br/>si now <= start - 24 h."
    note for User "blockedUntil puede ser nulo.<br/>Bloqueado si now < blockedUntil.<br/>Tercera falta:
bloquear 30 días desde now<br/>y poner absenceCount = 0."
    note for ReservationService "Sesión y rol verificados en cada operación.<br/>Aprobar: revalidar usuario,
equipo, estado y horario;<br/>comprobar y guardar de forma atómica.<br/>Cerrar una vez: APPROVED y now >=
end."
```

Criterios aplicados

Tomé los intervalos como [inicio, fin), por lo que dos reservas consecutivas pueden compartir la hora de fin/inicio. Solo las aprobadas ocupan equipo. La asistencia se registra por el administrador al terminar la hora; una cancelada no genera falta. Las tres faltas se acumulan desde el último bloqueo y el contador se reinicia al aplicar la sanción; el historial se conserva. Los 30 días empiezan al registrar la tercera falta y el usuario queda habilitado exactamente al cumplirse el plazo. El bloqueo no cancela automáticamente reservas previamente aprobadas. Todas las operaciones comprueban sesión y rol; la cancelación también comprueba propiedad. Estos detalles son supuestos de esta simulación porque la consigna no los define.

La validación de sintaxis y las capturas se realizan en un visor local con Mermaid 12.2.0. No se presentan como evidencia de revisión en mermaid.live. Los diagramas modelan reglas; no se implementó un sistema de reservas ni se probaron transacciones reales.

4. Capturas del antes y después

Las siguientes seis capturas muestran las versiones que se incluyen en la conversación. La primera conserva los fallos descritos y la segunda aplica los cinco cambios.

Las imágenes son capturas reales de Chromium renderizando los archivos Mermaid localmente. El nombre de cada archivo aparece en la captura. Los SVG originales permiten ampliar los diagramas con mayor nitidez y se pueden abrir desde `visor.html`.

Diagrama de flujo - Antes

Captura del visor local de Mermaid 12.2.0. Fuente: diagramas/antes/01_flujo.mmd.

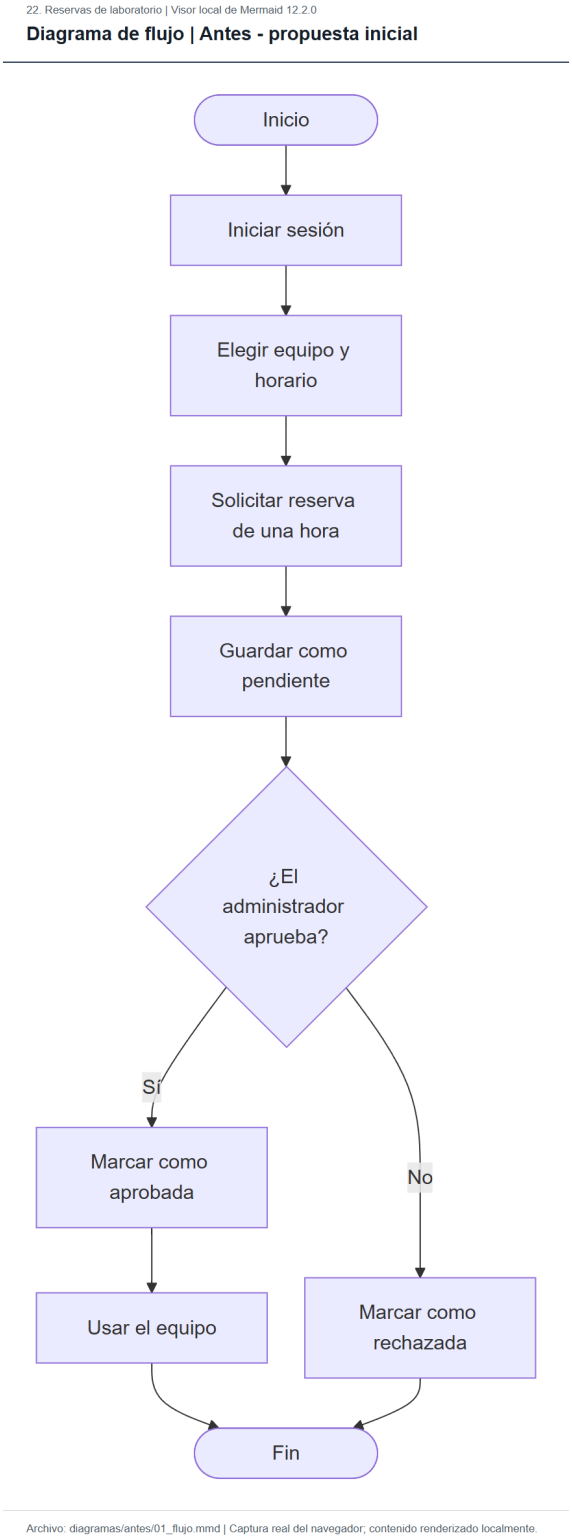
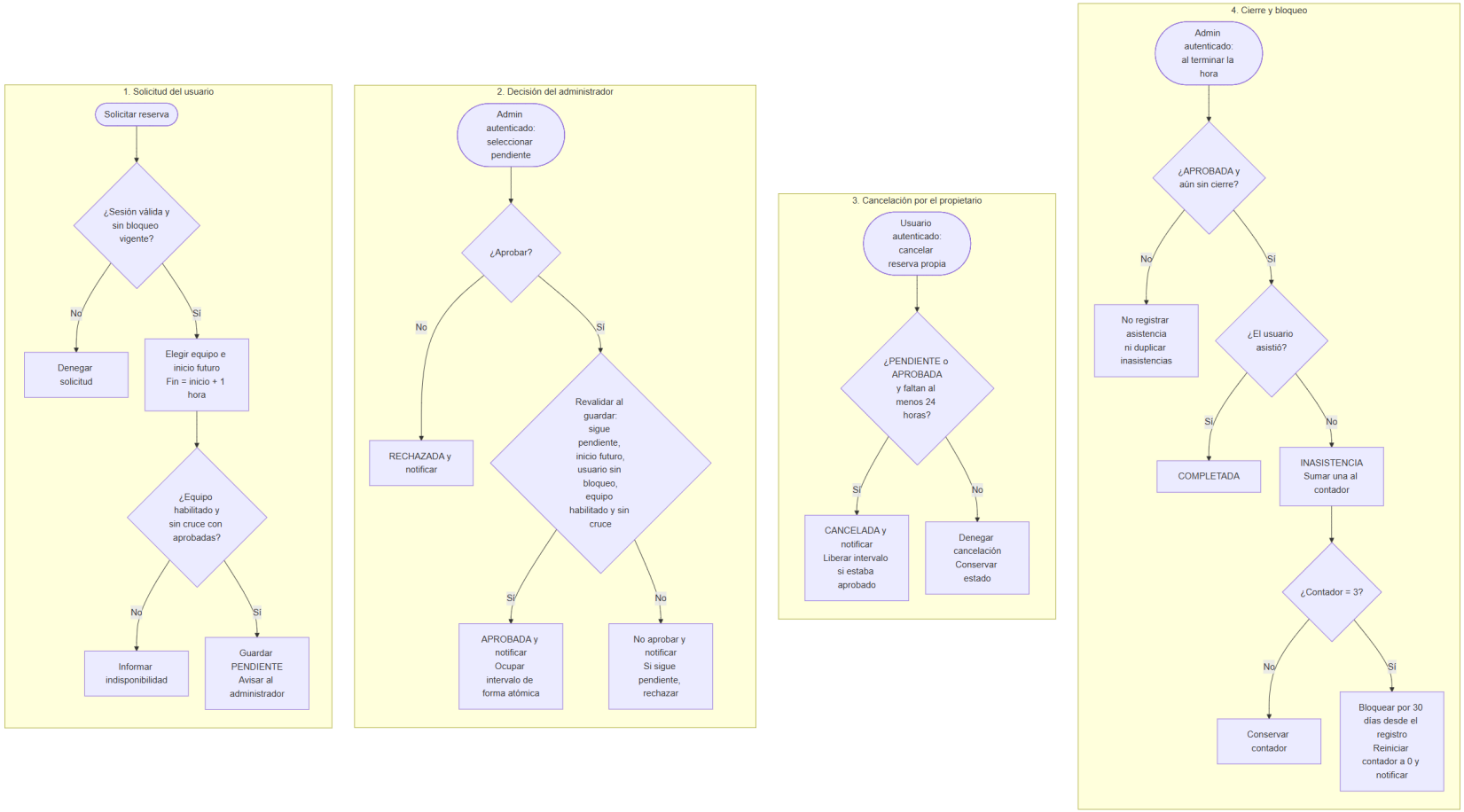


Diagrama de flujo - Después

Captura del visor local de Mermaid 12.2.0. Fuente: diagramas/despues/01_flujo.mmd.

22. Reservas de laboratorio | Visor local de Mermaid 12.2.0

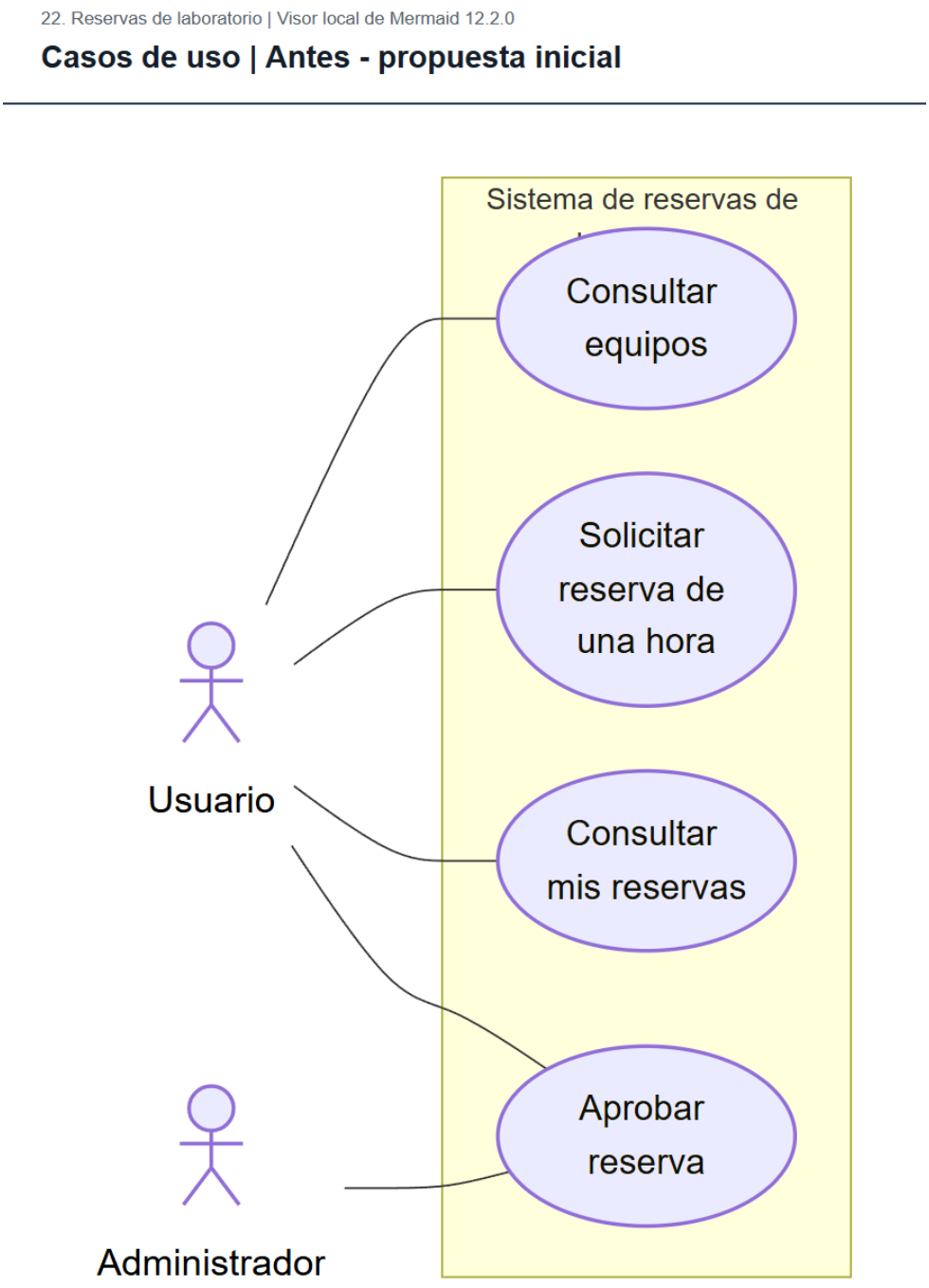
Diagrama de flujo | Después - versión corregida



Archivo: diagramas/despues/01_flujo.mmd | Captura real del navegador; contenido renderizado localmente.

Casos de uso - Antes

Captura del visor local de Mermaid 12.2.0. Fuente: diagramas/antes/02_casos_uso.mmd.



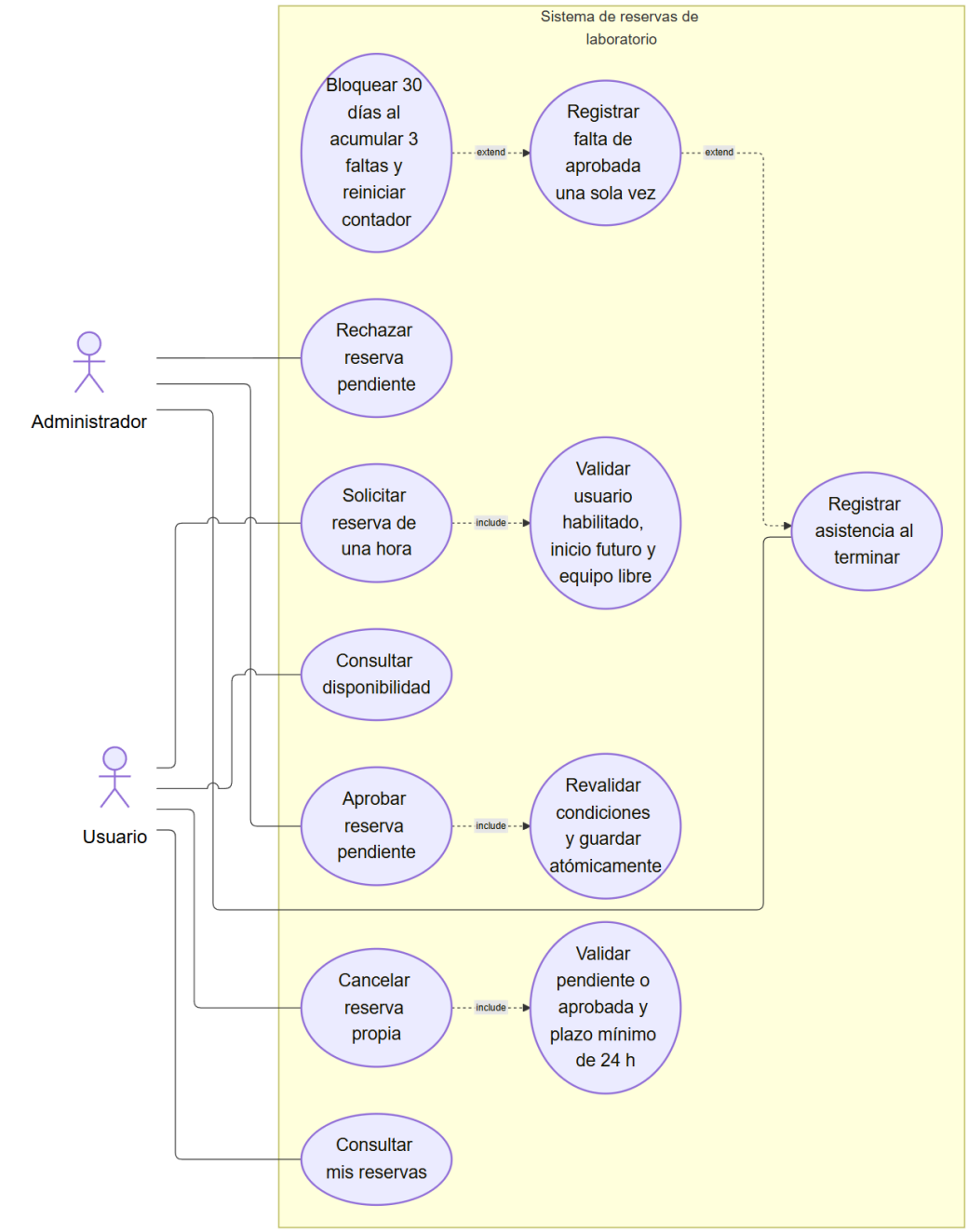
Archivo: diagramas/antes/02_casos_uso.mmd | Captura real del navegador; contenido renderizado localmente.

Casos de uso - Después

Captura del visor local de Mermaid 12.2.0. Fuente: diagramas/despues/02_casos_uso.mmd.

22. Reservas de laboratorio | Visor local de Mermaid 12.2.0

Casos de uso | Después - versión corregida



Archivo: diagramas/despues/02_casos_uso.mmd | Captura real del navegador; contenido renderizado localmente.

Diagrama de clases - Antes

Captura del visor local de Mermaid 12.2.0. Fuente: diagramas/antes/03_clases.mmd.

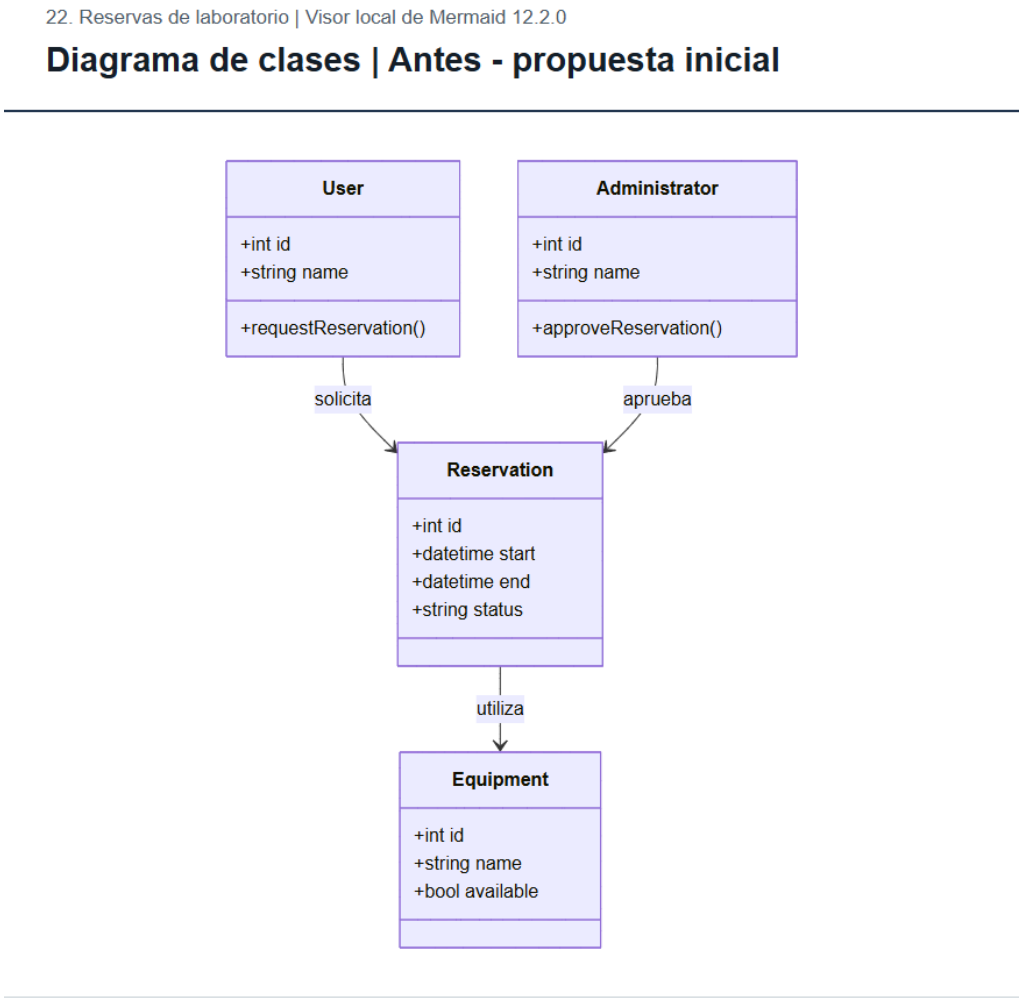
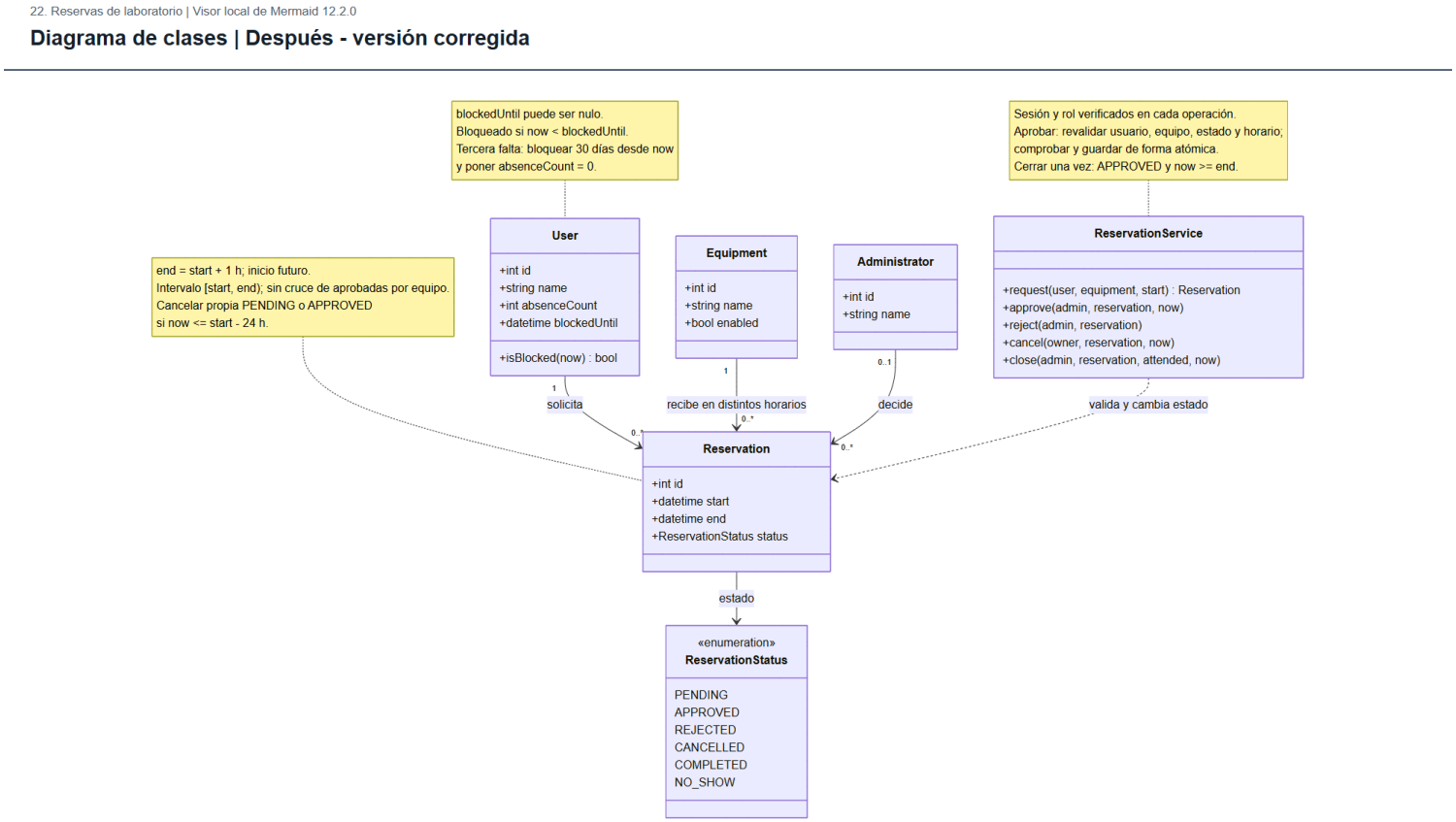


Diagrama de clases - Después

Captura del visor local de Mermaid 12.2.0. Fuente: diagramas/despues/03_clases.mmd.



5. Validación y referencias

- Motor: Mermaid 12.2.0, biblioteca oficial distribuida por jsDelivr.
- Los seis archivos pasaron `mermaid.parse()` y `mermaid.render()` sin errores.
- Las seis capturas PNG se tomaron de páginas reales en Chromium con el motor Mermaid local. Durante el renderizado se bloquearon las solicitudes externas.
- Cada SVG se conserva también por separado para ampliarlo sin perder nitidez.
- No se ejecutó OpenCode. No se subieron los diagramas a mermaid.live.
- Las comprobaciones de escritorio de `fallos_y_cambios.md` revisan coherencia de reglas; no son pruebas de una aplicación ni de concurrencia real.

Archivos comprobados

- `diagramas/antes/01_flujo.mmd`: sintaxis y renderizado correctos.
- `diagramas/antes/02_casos_uso.mmd`: sintaxis y renderizado correctos.
- `diagramas/antes/03_clases.mmd`: sintaxis y renderizado correctos.
- `diagramas/despues/01_flujo.mmd`: sintaxis y renderizado correctos.
- `diagramas/despues/02_casos_uso.mmd`: sintaxis y renderizado correctos.
- `diagramas/despues/03_clases.mmd`: sintaxis y renderizado correctos.

Límite de la evidencia

La revisión automática de permisos rechazó enviar el contenido local a Mermaid Live. Las capturas demuestran el renderizado local, no una visita a ese editor con los diagramas. Este paso de la consigna queda pendiente si se exige evidencia del sitio exacto. La revisión «sin IA» también requiere que el alumno haga su propia revisión.

Referencias técnicas

- Flujo: documentación oficial.
- Casos de uso nativos, Mermaid 12+.
- Clases: documentación oficial.
- Mermaid Live, para revisión manual si se autoriza.

Consultadas el 16 de octubre de 2026.